

BACKEND e DevOps

Desenvolvimento de Software para Web I

01 Web	02 Linguagens	03 Frameworks	04 APIs	05 Banco de Dados	06 Auth	07 Segurança	08 Arquitetura	09 Comunicação
10 API Gateway	11 Infra	12 DevOps	13 Performance	14 Testes	15 Observabilidade	16 Integrações	17 Boas Práticas	18 Tendências

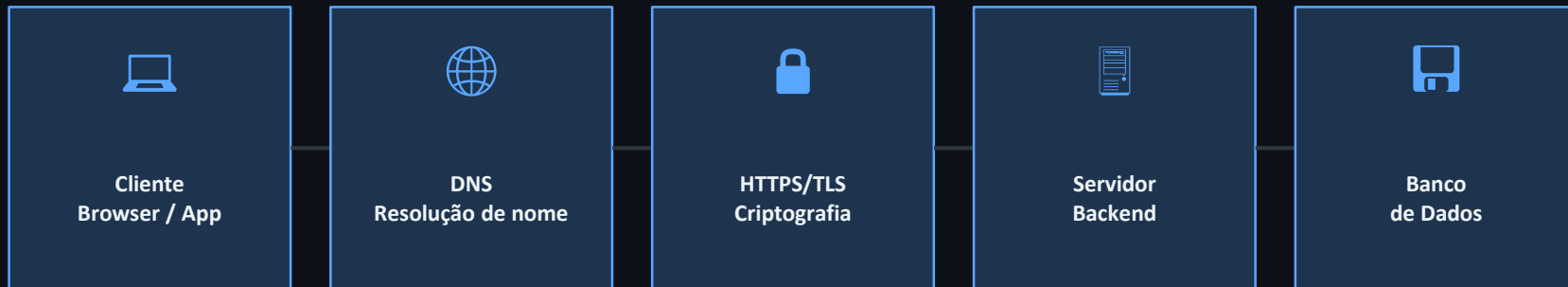


Fundamentos da Web

HTTP/HTTPS · Métodos · Status Codes · Headers · URL · Cookies · Sessions · CORS · REST ·
WebSockets

01

Arquitetura Cliente-Servidor



- Cliente sempre inicia a comunicação — servidor nunca inicia (exceto WebSockets)
- DNS resolve o nome do domínio (ex: api.site.com) para um endereço IP
- TLS faz o handshake: autentica o servidor e estabelece canal criptografado
- Servidor processa a lógica de negócio, consulta o banco e retorna a resposta

Métodos HTTP — O vocabulário da comunicação



GET

- Busca dados — nunca altera estado
- Idempotente: mesmo resultado N vezes
- Pode ser cacheado pelo browser/CDN
- Ex: GET /users/42



POST

- Cria novo recurso no servidor
- Não idempotente — duplica se repetido
- Retorna 201 Created com o novo recurso
- Ex: POST /users



PUT

- Substitui recurso inteiro — idempotente
- Envia representação completa
- Cria se não existir (alguns servidores)
- Ex: PUT /users/42



PATCH

- Atualiza parcialmente — só os campos enviados
- Mais eficiente que PUT para mudanças pequenas
- Ex: PATCH /users/42 { "name": "João" }



DELETE

- Remove recurso — idempotente
- Retorna 204 No Content em sucesso
- Ex: DELETE /users/42



OPTIONS

- Verifica métodos aceitos pelo servidor
- Usado automaticamente no preflight do CORS
- Não modifica estado

Status Codes HTTP — Guia Completo

Faixa	Código	Significado	Quando usar
2xx	200 OK	Sucesso	GET, PUT, PATCH bem-sucedidos
2xx	201 Created	Recurso criado	POST que cria um novo registro
2xx	204 No Content	Sucesso sem body	DELETE, PATCH sem retorno
3xx	301 Moved Perm.	Redirect permanente	URL mudou para sempre
3xx	304 Not Modified	Cache válido	ETag/If-None-Match match
4xx	400 Bad Request	Dados inválidos	Validação falhou, JSON malformado
4xx	401 Unauthorized	Não autenticado	Token ausente ou inválido
4xx	403 Forbidden	Sem permissão	Autenticado mas sem autorização
4xx	404 Not Found	Não encontrado	ID inexistente no banco
4xx	429 Too Many	Rate limit atingido	Muitas requisições em pouco tempo
5xx	500 Internal	Erro no servidor	Exceção não tratada no backend
5xx	503 Unavailable	Indisponível	Manutenção, sobrecarga, cold start

URL Anatomy — Toda parte tem um papel



Protocol

- `https://` → HTTP over TLS (criptografado)
- `http://` → texto puro — NUNCA em produção
- `wss://` → WebSocket seguro



Host + Port

- `api.example.com` → domínio ou subdomínio
- Porta 443: HTTPS padrão (omitida na URL)
- Porta 80: HTTP padrão (evitar em prod)



Path + Param

- `/users/42/orders` — rota da API
- `/users/:id` → parâmetro dinâmico na rota
- RESTful: substantivos no plural



Query Params

- `?page=2&limit=20&sort=-price`
- Filtros, paginação, ordenação — opcionais
- Nunca dados sensíveis em query string

Headers, Body, Cookies e Sessions



Request Headers

- Authorization: Bearer <jwt-token>
- Content-Type: application/json
- Accept: application/json
- X-Request-ID: <uuid> (tracing)
- Cache-Control: no-cache



Request Body

- Payload enviado em POST / PUT / PATCH
- Formato: JSON (mais comum), XML, form-data
- GET e DELETE não têm body (por convenção)
- ```
{ "name": "João", "email": "j@x.com" }
```



## Cookies

- Par chave=valor guardado no browser
- Enviado automaticamente em todo request
- Atributos: HttpOnly · Secure · SameSite
- Max-Age / Expires: validade do cookie



## Sessions

- Estado guardado no servidor (Redis/DB)
- Cookie só carrega o session ID (opaco)
- Stateful: requer session store compartilhado
- Alternativa moderna: JWT (stateless)

# Stateless vs Stateful

## ⚡ Stateless — REST

### Cada request é independente

Servidor não guarda estado entre requests

### Escala horizontal naturalmente

Qualquer instância serve qualquer request

### JWT / API Keys

Autenticação embutida no próprio token

### Cache-friendly

CDNs e proxies podem cachear livremente

### Exemplos

REST APIs — Netflix, GitHub, AWS, Twitter

## 🍪 Stateful — Sessions

### Estado mantido no servidor

Session store compartilhado (Redis)

### Mais difícil de escalar

Exige sticky sessions ou replicação

### Session Cookie

Browser envia session ID em cada request

### Mais simples para certos fluxos

Carrinhos de compra, wizards multi-passo

### Exemplos

Sistemas bancários tradicionais, e-commerce antigo



# CORS — Cross-Origin Resource Sharing



## O problema

- Browser bloqueia requests cross-origin por segurança
- api.outro.com bloqueado quando chamado de app.com
- Política SOP: Same-Origin Policy do browser



## Preflight Request

- Browser envia OPTIONS antes do request real
- Pergunta: 'Posso fazer POST deste origin?'
- Servidor responde com headers de permissão



## Headers CORS

- Access-Control-Allow-Origin: https://app.com
- Access-Control-Allow-Methods: GET,POST,PUT
- Access-Control-Allow-Headers: Authorization
- Access-Control-Max-Age: 86400 (cache preflight)



## Config no servidor

- Middleware de CORS no Express/NestJS/Django
- NUNCA usar wildcard \* com credenciais
- Whitelist de origens permitidas explícita
- Erros de CORS = problema de servidor, não de cliente

# REST vs WebSockets

## REST — Request / Response

### Cliente sempre inicia

Servidor só responde quando chamado

### HTTP stateless

Cada request carrega tudo que precisa

### Cache-friendly

GET responses podem ser cacheadas

### Simples de escalar

Load balancer distribui facilmente

### Ideal para

CRUD, consultas, operações pontuais

## WebSockets — Full-Duplex

### Conexão TCP persistente

Handshake único via HTTP Upgrade

### Bidirecional

Servidor envia dados SEM request do cliente

### Baixa latência

Sem overhead de novo handshake a cada msg

### Mais complexo de escalar

Sticky sessions ou broker (Redis Pub/Sub)

### Ideal para

Chat, jogos, dashboards live, trading, colaboração

# Linguagens de Programação Backend

JS/TS · Python · Java · C# · PHP · Go · Ruby · Tipos · Async · Threads · Erros · Módulos

# Principais Linguagens Backend (parte 1)



## JavaScript / TypeScript

- Runtime: Node.js (V8) — event loop não-bloqueante
- TypeScript: tipagem estática, IntelliSense, refactor seguro
- npm/yarn: maior ecossistema de packages do mundo
- Ideal: APIs REST, real-time, serverless, BFF



## Python

- Legível e expressivo — curva de aprendizado baixa
- Frameworks: FastAPI, Django, Flask
- Forte em Data Science, ML e AI backends
- pip / Poetry — ecossistema científico maduro



## Java

- JVM: Write once run anywhere — alto desempenho
- Spring Boot: maior ecossistema enterprise do mercado
- Fortemente tipado — refactor seguro em projetos grandes
- Maven / Gradle — CI/CD maduro e tooling excelente



## C# (.NET)

- Microsoft — ASP.NET Core cross-platform (Linux/Mac/Win)
- LINQ: queries expressivas sobre coleções e DBs
- Azure native — integração nativa com serviços cloud
- Performance comparável a Go em benchmarks recentes

# Principais Linguagens Backend (parte 2)



## PHP

- Linguagem que construiu a web — WordPress, Magento
- Laravel: MVC moderno, Eloquent ORM, artisan CLI
- Composer: gerenciador de dependências maduro
- Serverless via Bref — executa PHP em AWS Lambda



## Go (Golang)

- Compilado para binário único — sem runtime externo
- Goroutines: concorrência nativa, leve e barata
- Throughput altíssimo — ideal para microserviços
- Kubernetes, Docker, Terraform — escritos em Go



## Ruby

- Expressividade máxima — 'código feliz de escrever'
- Rails: convention over config — produtividade extrema
- Gems: ecossistema rico de bibliotecas prontas
- Ideal: startups, protótipos, MVPs rápidos



## Rust

- Memory-safe sem garbage collector — zero crashes de memória
- Zero-cost abstractions — C++ performance com safety
- Ideal: sistemas, CLIs, WebAssembly, edge functions
- Adoção crescente: Linux kernel, Windows, Firefox

# Tipos de Dados e Controle de Fluxo



## Tipos Primitivos

- Integer: int8/16/32/64, uint — tamanhos específicos
- Float: float32/float64, decimal (dinheiro!)
- String: imutável na maioria das linguagens
- Boolean: true/false — null/nil/None para ausência



## Tipos Compostos

- Array/List: sequência ordenada com índice
- Object/Dict/Map: chave-valor — busca O(1)
- Set: valores únicos — interseção e união rápidos
- Struct/Class: agrupar dados e comportamento



## Controle de Fluxo

- if/else · switch/match — ramificação condicional
- for/while/do-while — repetição com condição
- break · continue · return — controle de iteração
- Pattern matching (Rust, Kotlin, Python 3.10+)



## Tratamento de Erros

- try/catch/finally — Java, JS, Python, C#
- Result<T,E> — Rust: obriga a tratar o erro
- Error values — Go: return value, err := fn()
- NUNCA ignorar erros: catch vazio = bug oculto

# Programação Assíncrona — Callbacks → Async/Await

```
// ✗ Callback Hell — difícil de ler e manter
getData(function(data) {
 getUser(data.id, function(user) {
 getOrders(user.id, function(orders) {
 console.log(orders); // Callback hell!
 });
 });
});

// ✔ Async/Await — legível e com tratamento de erros
async function loadUserOrders(userId: string) {
 try {
 const user = await userRepo.findById(userId);
 const orders = await orderRepo.findByUser(user.id);

 // Promise.all: executa em PARALELO quando independentes
 const [inventory, pricing] = await Promise.all([
 inventoryService.check(orders),
 pricingService.calculate(orders),
]);
 return { orders, inventory, pricing };
 } catch (err) {
 logger.error({ userId, err }, "Failed to load orders");
 throw new AppError("Could not load orders", 500);
 }
}
```

## Event Loop (Node.js)

Single thread com I/O não-bloqueante — libuv lida com I/O em paralelo

## Promises

Representam valor futuro — `.then().catch()` ou `async/await`

## Promise.all()

Executa em paralelo — falha se qualquer um falhar

## Promise.allSettled()

Executa todos — retorna resultado de cada um (sucesso ou falha)

## Python asyncio

`async def` / `await` / `asyncio.gather()` — corrotinas nativas

## Go goroutines

`go fn()` — thread leve; channels para comunicação segura

# Threads e Concorrência por Linguagem



## Java — Threads reais

- `java.lang.Thread` — thread do sistema operacional
- `ExecutorService` — pool de threads gerenciado
- Virtual Threads (Java 21) — leve como corrotinas
- `synchronized`, `locks`, `atomic` — controle de acesso



## Go — Goroutines

- `go func()` — lança goroutine em microsegundos
- Scheduler M:N — mapeia goroutines em OS threads
- Channels — comunicação segura entre goroutines
- `WaitGroup` — aguarda grupo de goroutines terminar



## Python — GIL

- GIL: Global Interpreter Lock — 1 thread por vez no CPython
- `threading`: concorrência para I/O, não para CPU
- `multiprocessing`: processos reais — bypass do GIL
- `asyncio`: solução recomendada para I/O-bound tasks



## Node.js — Event Loop

- Single thread — mas I/O é sempre não-bloqueante
- Worker Threads — módulo para CPU-intensive tasks
- Cluster — múltiplos processos Node numa máquina
- `libuv` gerencia thread pool interno (file, DNS, crypto)





# Frameworks Backend

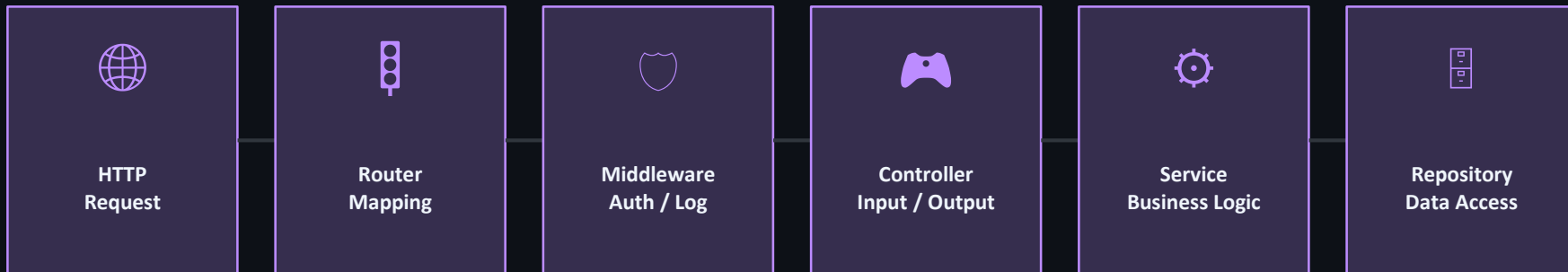
Express · NestJS · Django · Flask · FastAPI · Spring Boot · ASP.NET · Laravel · Rails

03

## Frameworks — Visão Geral Comparativa

| Framework     | Linguagem  | Estilo            | Ponto forte                                 |
|---------------|------------|-------------------|---------------------------------------------|
| Express.js    | Node.js    | Minimalista       | Flexibilidade total — middleware chain      |
| NestJS        | TypeScript | Opinionado + DI   | Arquitetura robusta, testável, enterprise   |
| Django        | Python     | Full-stack / MTV  | Admin automático, ORM, baterias inclusas    |
| Flask         | Python     | Micro             | Leveza, flexibilidade, prototipagem         |
| FastAPI       | Python     | Async + OpenAPI   | Pydantic, validação automática, alta perf   |
| Spring Boot   | Java       | Enterprise + DI   | Ecosistema maduro, JPA, microserviços       |
| ASP.NET Core  | C#         | MVC + Minimal API | Cross-platform, performance, Azure native   |
| Laravel       | PHP        | Full-stack / MVC  | Eloquent ORM, artisan CLI, ecossistema rico |
| Ruby on Rails | Ruby       | Full-stack / MVC  | Convention over config — produtividade max  |

## Ciclo de uma Requisição — MVC



- Router: mapeia URL + Método HTTP → Controller correto
- Middleware: intercepta antes/depois do controller (auth, log, rate limit, CORS, validação global)
- Controller: valida input, chama Service, serializa e retorna response
- Service: lógica de negócio, orquestra operações — stateless, testável
- Repository: abstrai o banco — queries, CRUD — o controller nunca acessa o DB diretamente

# NestJS — Controller + Service + DI

```
// users.controller.ts
@Controller('users')
@UseGuards(JwtAuthGuard)
export class UsersController {
 constructor(private readonly svc: UsersService) {}

 @Get()
 findAll(@Query() q: PaginationDto) {
 return this.svc.findAll(q);
 }

 @Get(':id')
 findOne(@Param('id', ParseUUIDPipe) id: string) {
 return this.svc.findOneOrThrow(id);
 }

 @Post()
 @HttpCode(201)
 create(@Body() dto: CreateUserDto) {
 return this.svc.create(dto);
 }
}

// users.service.ts
@Injectable()
export class UsersService {
 constructor(
 @InjectRepository(User)
 private readonly repo: Repository<User>,
) {}

 async create(dto: CreateUserDto) {
 const user = this.repo.create(dto);
 return this.repo.save(user);
 }
}
```

## @Controller

Define o prefixo de rota — /users

## @UseGuards

Middleware de autenticação aplicado ao grupo

## @Get / @Post

Mapeia método HTTP ao handler

## @Param / @Body

Injeta dados do request automaticamente

## ParseUUIDPipe

Valida e transforma parâmetro antes do handler

## @Injectable + DI

IoC container gerencia ciclo de vida — facilita testes

# Middlewares — O coração dos frameworks



## Autenticação

- Valida JWT/session antes de qualquer handler
- Rejeita com 401 se token inválido ou ausente
- Popula req.user com dados do token para uso posterior



## Logging & Tracing

- Registra: método, path, status code, latência
- Gera e propaga X-Request-ID para tracing distribuído
- Pino / Winston — logs estruturados em JSON



## Rate Limiting

- Limita requests por IP / usuário / API key
- Retorna 429 Too Many Requests ao exceder limite
- express-rate-limit, Nginx limit\_req, API Gateway



## Validação Global

- Valida body/params antes do controller receber
- Retorna 400 com detalhes de cada campo inválido
- NestJS ValidationPipe · Django serializers · Pydantic



## Clean Architecture

- Camadas: Presentation → Application → Domain → Infra
- Domain não depende de frameworks ou banco
- Facilita trocar Express por Fastify sem tocar negócio
- Testabilidade: mockar infra sem mudar domain



## Injeção de Dependência

- IoC container cria e injeta instâncias automaticamente
- Scopes: Singleton · Scoped (por request) · Transient
- NestJS / Spring / ASP.NET têm DI nativo
- Facilita testes: substituir implementação por mock



## Config por Ambiente

- .env para desenvolvimento local
- CI/CD secrets para staging e produção
- 12-factor App: externalizar toda configuração
- Nunca hardcodar URLs, senhas ou API keys no código



## Repositories e Services

- Repository: acesso ao DB — CRUD e queries específicas
- Service: lógica de negócio, orquestra repositories
- Controller nunca acessa DB diretamente
- Uma responsabilidade por classe — SOLID S



# Design de APIs

REST · GraphQL · gRPC · Endpoints · Versionamento · Paginação · Validação · OpenAPI

04

# REST — Design de Endpoints

```
✔ Boas práticas de endpoints RESTful
GET /api/v1/users # listar com paginação
GET /api/v1/users/:id # buscar por ID
POST /api/v1/users # criar → retorna 201
PATCH /api/v1/users/:id # atualizar parcialmente
DELETE /api/v1/users/:id # remover → retorna 204

Recursos aninhados (quando o contexto faz sentido)
GET /api/v1/users/:id/orders
POST /api/v1/users/:id/orders

Filtros, paginação offset e ordenação
GET /products?category=tech&minPrice=100&sort=-price&page=2&limit=20

Paginação cursor-based (grande volume de dados)
GET /posts?after=eyJpZCI6MTAwfQ==&limit=20
Response: { data: [...], nextCursor: "eyJpZCI6MTIwfQ==" }

Versionamento — nunca quebrar clientes existentes
GET /api/v1/users → comportamento original
GET /api/v2/users → novo shape de resposta

✗ Evitar: verbos na URL e métodos incorretos
GET /getUser | POST /deleteUser | GET /user_list
```

## Substantivos, não verbos

URL = recurso. Método HTTP = ação sobre ele

## Plural nos recursos

/users, /products, /orders — consistência

## Versionamento

/api/v1/ na URL — não quebra clientes antigos

## Filtros via query params

?filter=value&sort=field&page=N&limit=N

## Paginação cursor-based

Mais eficiente que offset para grandes volumes

## Status codes corretos

201, 204, 400, 401, 403, 404, 429, 500





## Um único endpoint

- POST /graphql para toda a API
- Schema fortemente tipado — contrato explícito
- Introspection: cliente descobre o schema



## Sem over/under-fetching

- Cliente pede exatamente os campos que precisa
- Evita N chamadas REST para montar uma tela
- Ideal para apps com UIs complexas e variadas



## Subscriptions

- WebSocket para dados em tempo real
- Cliente subscreve a eventos do servidor
- Ex: notificações, feeds ao vivo, colaboração



## Performance

- DataLoader: batch de queries — resolve N+1
- Persisted queries: cache de queries frequentes
- Depth limiting: previne queries abusivas recursivas



## gRPC — Alta Performance

- Protocol Buffers: serialização binária (10x < JSON)
- HTTP/2: streaming, multiplexing, header compression
- Schema .proto: tipagem forte e geração de código
- Ideal: comunicação interna entre microserviços



## gRPC Streaming Types

- Unary: request/response tradicional
- Server-side: servidor envia stream de respostas
- Client-side: cliente envia stream de requests
- Bidirectional: ambos streamam ao mesmo tempo



## OpenAPI / Swagger

- Especificação YAML/JSON do contrato da API
- Swagger UI: interface interativa para testar
- Gera SDKs em múltiplas linguagens automaticamente
- Contrato vivo entre front-end e back-end



## Validação e Serialização

- Validar: tipo, formato, obrigatoriedade, regras
- JS: Zod, Joi · Python: Pydantic · Java: Bean Validation
- DTOs: expor só o que o cliente precisa ver
- Excluir campos sensíveis (password, secret) nas responses

# REST vs GraphQL vs gRPC — Quando usar cada um

| Aspecto     | REST             | GraphQL             | gRPC                       |
|-------------|------------------|---------------------|----------------------------|
| Protocolo   | HTTP 1.1 / 2     | HTTP 1.1 / 2        | HTTP/2                     |
| Formato     | JSON / XML       | JSON                | Protocol Buffers (binário) |
| Endpoints   | Múltiplos        | Um (/graphql)       | Por método (.proto)        |
| Fetching    | Fixed response   | Flexível            | Fixed por método           |
| Tipo-safe   | Manual (OpenAPI) | Schema nativo       | Forte (.proto)             |
| Streaming   | SSE / WS à parte | Subscriptions (WS)  | Nativo (HTTP/2)            |
| Performance | Boa              | Boa + DataLoader    | Excelente                  |
| Ideal para  | APIs públicas    | BFF, apps complexas | Microserviços internos     |

# Banco de Dados

SQL · NoSQL · Modelagem · ACID · Índices · Joins · ORM · Prisma · TypeORM · Hibernate

05

# Bancos Relacionais (SQL) vs NoSQL

## Bancos Relacionais — SQL

### PostgreSQL

Open-source, extensível, JSON nativo, melhor do mundo

### MySQL / MariaDB

Popular, web-friendly, RDS na AWS

### SQL Server

Microsoft, integrado com .NET e Azure

### SQLite

File-based, embarcado, ideal para dev e mobile

### Schema rígido

Tabelas, colunas tipadas, relações explícitas

### ACID garantido

Consistência forte em transações complexas

## NoSQL — Flexibilidade e Escala

### MongoDB

Documentos JSON, schema flexível, Atlas cloud

### Redis

In-memory: cache, pub/sub, sessions, rate limit

### Cassandra

Wide-column, escrita massiva, multi-datacenter

### DynamoDB

AWS serverless, key-value, auto-scale ilimitado

### Neo4j

Grafo — relacionamentos complexos, Cypher query

### BASE theorem

Basically Available, Eventual Consistency



## Chaves e Relações

- PK: identificador único de cada registro
- FK: referencia PK de outra tabela
- Relação 1:1 — usuário ↔ perfil
- Relação 1:N — usuário → pedidos
- Relação N:M — produtos ↔ categorias (pivot table)



## Normalização

- 1NF: sem grupos repetidos, valores atômicos
- 2NF: dependência total da chave primária
- 3NF: sem dependências transitivas
- Desnormalizar para performance quando necessário



## Índices

- B-Tree: padrão — busca, range, ORDER BY
- Hash: igualdade exata — chaves de cache
- GIN: arrays, JSONB, full-text search
- BRIN: colunas sequenciais (datas, IDs crescentes)
- CREATE INDEX CONCURRENTLY — sem bloquear tabela



## Análise de Queries

- EXPLAIN ANALYZE: plano de execução real
- Seq Scan → precisa de índice!
- Index Scan → ótimo
- N+1 Problem: usar JOIN ao invés de loop de queries

# SQL — Joins, Transações e Índices

```
-- JOIN com agregação e filtros
SELECT u.name,
 COUNT(o.id) AS total_orders,
 SUM(o.total) AS revenue
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE u.created_at > NOW() - INTERVAL '90 days'
GROUP BY u.id, u.name
HAVING COUNT(o.id) > 5
ORDER BY revenue DESC
LIMIT 20;

-- Transação ACID — tudo ou nada
BEGIN;
 UPDATE accounts SET balance = balance - 500
 WHERE id = 1;
 INSERT INTO transactions (from_id, to_id, amount)
 VALUES (1, 2, 500);
 UPDATE accounts SET balance = balance + 500
 WHERE id = 2;
COMMIT; -- ROLLBACK automático se qualquer UPDATE falhar

-- Índice composto sem bloquear produção
CREATE INDEX CONCURRENTLY idx_orders_user_date
 ON orders (user_id, created_at DESC);
```

## LEFT JOIN

Inclui usuários sem pedidos (valor NULL)

## HAVING

Filtra após agregação — WHERE filtra antes

## Transação ACID

Atomicidade: saldo nunca fica inconsistente

## ROLLBACK auto

Erro em qualquer statement desfaz tudo

## CONCURRENTLY

Cria índice sem lock — tabela continua online

## Composite index

(user\_id, created\_at) cobre WHERE + ORDER em 1 index

# Propriedades ACID — Garantias do Banco

| Propriedade     | O que garante                                      | Exemplo prático                                             |
|-----------------|----------------------------------------------------|-------------------------------------------------------------|
| Atomicity (A)   | Tudo ou nada — nenhuma operação parcial            | Transferência bancária: debita E credita ou nenhum dos dois |
| Consistency (C) | Dados sempre em estado válido (constraints ativos) | Saldo nunca negativo se existe CHECK (balance >= 0)         |
| Isolation (I)   | Transações paralelas não interferem entre si       | Dois usuários editando o mesmo produto — sem conflito       |
| Durability (D)  | Dados commitados sobrevivem a crashes              | Transação confirmada existe mesmo após queda de energia     |



# ORMs e ODMs — Abstração do Banco



## Prisma (Node/TS)

- Schema declarativo → types TypeScript automáticos
- Migrações versionadas, Prisma Studio (GUI)
- Type-safe: erro de campo detectado em compile-time
- Suporta PostgreSQL, MySQL, SQLite, MongoDB, SQL Server



## TypeORM (Node/TS)

- Decorators: @Entity, @Column, @ManyToOne
- ActiveRecord ou DataMapper pattern
- Migrações, relations lazy/eager, query builder
- Suporta a maioria dos bancos relacionais



## Hibernate / JPA (Java)

- Padrão JPA — portátil entre providers
- HQL/JPQL: queries orientadas a objeto
- Lazy/eager loading, cache de segundo nível
- Spring Data JPA: findByEmail, findAllByStatus...



## Entity Framework (.NET)

- Code-first: entidades → migrations → banco
- LINQ: queries type-safe sobre entidades
- ChangeTracker: rastreia mudanças automaticamente
- Suporta PostgreSQL, MySQL, SQL Server, SQLite



## SQLAlchemy (Python)

- ORM + Core (SQL puro com abstração)
- Alembic: migrações versionadas
- Sessões: Unit of Work pattern
- Flask-SQLAlchemy, FastAPI + SQLAlchemy



## Mongoose (MongoDB)

- ODM para MongoDB — schemas e validação
- Middleware pre/post hooks (password hash!)
- populate(): equivalente a JOIN no MongoDB
- Suporta TypeScript nativamente

**ORMs (Object-Relational Mapping) - SQL**  
**ODMs (Object-Document Mapping) – NoSQL**  
<https://db-engines.com/en/>

# Autenticação e Autorização

Login · bcrypt · argon2 · JWT · OAuth 2.0 · OpenID Connect · RBAC · ABAC

# AuthN vs AuthZ — Conceitos Fundamentais

## 🔑 Autenticação (AuthN) — Quem é você?

### Verifica identidade

Prova que você é quem diz ser

### Login + senha

Hash comparado contra banco de dados

### MFA / 2FA

TOTP (Google Auth), SMS, WebAuthn (passkeys)

### SSO

Login único para múltiplos sistemas

### OAuth 2.0 + OIDC

Login social — Google, GitHub, Microsoft

### Resultado

Token ou sessão confirmando identidade

## 🛡️ Autorização (AuthZ) — O que pode fazer?

### Verifica permissões

O que esse usuário autenticado pode fazer?

### RBAC

Role-Based: admin, editor, viewer, guest

### ABAC

Attribute-Based: regras contextuais complexas

### Resource-level

User só edita o próprio perfil — verificar ownership

### Sempre no backend

Nunca confiar só no front-end para autorização!

### Resultado

403 Forbidden se não autorizado

# Hash de Senhas — Nunca Texto Puro



## bcrypt

- Algoritmo adaptativo — cost factor (rounds) ajustável
- \$2b\$12\$... — 12 rounds → ~250ms por hash
- Inclui salt automático — sem rainbow tables
- `bcrypt.compare()` — comparação segura em tempo constante



## argon2

- Vencedor do Password Hashing Competition 2015
- `argon2id`: resistente a GPU e side-channel attacks
- Parâmetros: memory, iterations, parallelism
- Recomendação OWASP atual para novas aplicações



## Nunca MD5/SHA1

- Algoritmos de velocidade — não de senha!
- Quebráveis com GPU em segundos (brute force)
- Rainbow tables prontas para hashes comuns
- Mesmo SHA-256/512 são rápidos demais para senhas



## Salt automático

- `bcrypt` e `argon2` geram salt único por senha
- Mesmo password → hash diferente cada vez
- Elimina ataques de rainbow table
- Salt fica embutido na string de hash

# JWT — Fluxo Completo de Autenticação

```
// 1. Login: valida credenciais e emite tokens
async function login(email: string, password: string) {
 const user = await userRepo.findByEmail(email);
 if (!user) throw new UnauthorizedException('Invalid credentials');

 const valid = await bcrypt.compare(password, user.passwordHash);
 if (!valid) throw new UnauthorizedException('Invalid credentials');

 const payload = { sub: user.id, role: user.role };
 return {
 // Vida curta — usado em todo request
 accessToken: jwt.sign(payload, JWT_SECRET, { expiresIn: '15m'
 }},
 // Vida longa — troca o access token quando expira
 refreshToken: jwt.sign(payload, REFRESH_SECRET, { expiresIn: '7d' }),
 };
}

// 2. Middleware: valida access token em cada request
function jwtGuard(req, res, next) {
 const token = req.headers.authorization?.split(' ')[1]; // Bearer
 <token>
 try {
 req.user = jwt.verify(token, JWT_SECRET);
 next();
 } catch {
 res.status(401).json({ error: 'Token inválido ou expirado' });
 }
}
```

## JWT = 3 partes

header.payload.signature — base64url — NÃO é criptografado!

## Access Token (15m)

Vida curta — enviado no header Authorization: Bearer <token>

## Refresh Token (7d)

Vida longa — guardado em HttpOnly cookie para renovar access

## Claims no payload

sub (userId), role, email, iat (issued at), exp (expiry)

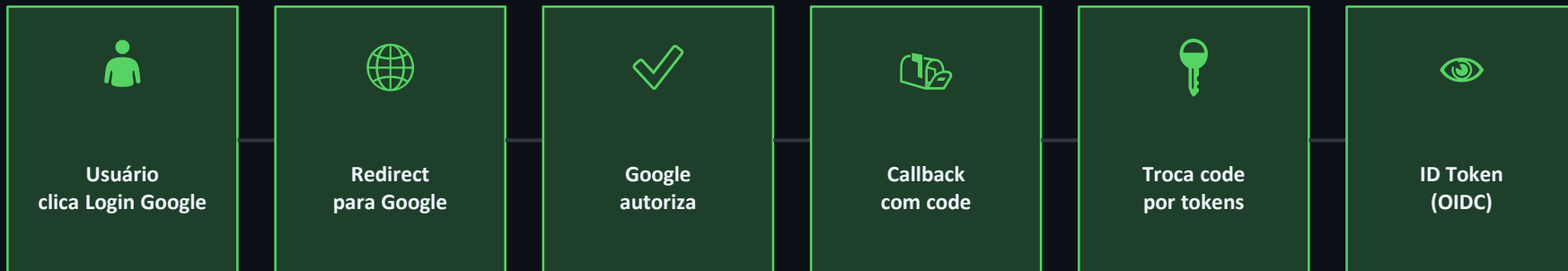
## Nunca senha no JWT

Payload é decodificável por qualquer um — só dados não-sensíveis

## Revogação de JWT

Blacklist no Redis ou refresh token rotation

## OAuth 2.0 + OpenID Connect — Fluxo



- OAuth 2.0: protocolo de DELEGAÇÃO de acesso — não de autenticação!
- OpenID Connect (OIDC): camada de identidade SOBRE OAuth 2.0 — adiciona ID Token (JWT com dados do usuário)
- PKCE: Proof Key for Code Exchange — obrigatório para SPAs e mobile apps (previne code interception)
- Access Token: para chamar APIs · ID Token: identidade do usuário · Refresh Token: renovar access

# RBAC vs ABAC e Scopes



## RBAC — Role-Based

- Papéis: admin, editor, viewer, moderator
- Usuário tem um ou mais roles
- Simples de implementar e auditar
- Ex: admin pode DELETE, viewer só GET



## ABAC — Attribute-Based

- Regras por atributos contextuais
- Ex: user.dept === resource.dept AND time < 18h
- Muito flexível, ideal para sistemas enterprise
- Implementado com policy engines (OPA, Casbin)



## RBAC no JWT

- Role embutida no payload do JWT
- Middleware verifica antes do controller
- @Roles('admin') no NestJS
- @PreAuthorize('has Role("ADMIN")') no Spring



## OAuth Scopes

- Permissões granulares no access token
- read:users · write:orders · admin:all
- Cliente solicita apenas scopes necessários
- Resource server valida os scopes do token



# Segurança Backend

OWASP Top 10 · SQL Injection · XSS · CSRF · HTTPS/TLS · Rate Limiting · Secrets

07



# OWASP Top 10 — Vulnerabilidades Críticas (parte 1)



## A01 — Injection

- SQL, NoSQL, Command, LDAP Injection
- Nunca concatenar input em queries ou comandos
- Usar prepared statements / ORM / parameterized
- Ex perigoso: WHERE id=input\_direto — injetável



## A02 — Broken Auth

- Senhas fracas sem rate limit no login
- Brute force sem bloqueio de IP/conta
- Fix: bcrypt + rate limit + MFA obrigatório
- Session fixation — regenerar ID após login



## A03 — Sensitive Data

- Dados sensíveis sem criptografia em trânsito
- Senhas em texto puro no banco de dados
- Fix: HTTPS obrigatório + bcrypt/argon2
- AES-256 para dados sensíveis em repouso



## A05 — Broken Access Ctrl

- IDOR: acessar recurso de outro usuário
- Ex: GET /users/42 sem verificar se é o próprio user
- Fix: verificar ownership em cada endpoint
- Testes automáticos de autorização no pipeline

# OWASP Top 10 — Vulnerabilidades Críticas (parte 2)



## A06 — Misconfiguration

- Headers de segurança ausentes
- Debug mode / stack traces em produção
- Credenciais padrão não alteradas
- Fix: Helmet.js, security hardening checklist



## A07 — XSS

- Script malicioso executado no browser da vítima
- Tipos: Reflected, Stored, DOM-based
- Fix: sanitizar output, Content-Security-Policy
- DOMPurify para HTML dinâmico no front



## A08 — CSRF

- Request forjado enviado de outro site
- Explora cookies enviados automaticamente
- Fix: SameSite=Strict cookies + CSRF token
- Double Submit Cookie ou Synchronizer Token



## A09 — Vulnerable Components

- Dependências com CVEs conhecidas
- Fix: npm audit / pip-audit / Snyk / Dependabot
- Atualizar dependências regularmente
- Software Composition Analysis (SCA) no CI

# Proteções Essenciais na Prática

```
// 1. Helmet.js – security headers em 1 linha
app.use(helmet());
// Adiciona: X-Frame-Options, Content-Security-Policy,
// Strict-Transport-Security (HSTS), X-XSS-Protection, etc.

// 2. Rate Limiting – login: 10 tentativas / 15 min
const loginLimiter = rateLimit({
 windowMs: 15 * 60 * 1000,
 max: 10,
 skipSuccessfulRequests: true,
 message: { error: 'Too many login attempts' }
});
app.use('/auth/login', loginLimiter);

// 3. SQL Injection – SEMPRE parâmetros separados
// ✗ NUNCA: "SELECT * WHERE id=" + req.params.id
// ✔ SEMPRE:
const user = await db.query(
 'SELECT * FROM users WHERE id = $1 AND active = true',
 [req.params.id] // banco trata como dado, não como SQL
);

// 4. Validação com Zod – rejeita no tipo
const CreateUserSchema = z.object({
 email: z.string().email().max(255),
 name: z.string().min(2).max(100).trim(),
 password: z.string().min(8).max(128),
});
```

## Helmet.js

Configura +11 security headers com 1 linha de código

## Rate Limiting

express-rate-limit: limita por IP, usuário ou API key

## Prepared Statements

Banco interpreta input como dado — impossível injetar SQL

## Input Validation

Zod / Joi / Pydantic: validar e sanitizar TUDO vindo de fora

## HTTPS obrigatório

HTTP→HTTPS redirect + HSTS para forçar no browser

## Secrets no env

Nunca em código — .env local, Vault / Secrets Manager em prod



## HTTPS / TLS

- TLS 1.2+ obrigatório — TLS 1.0/1.1 deprecated
- Let's Encrypt: certificados gratuitos e automáticos
- HSTS: força HTTPS por 1 ano no browser
- SSL termination no load balancer ou Nginx



## Gestão de Secrets

- HashiCorp Vault: secrets dinâmicos com rotação
- AWS Secrets Manager / GCP Secret Manager
- Nunca commitar .env no Git (git-secrets, .gitignore)
- CI/CD secrets: GitHub Actions, GitLab CI variables



## Anti Brute Force

- Rate limit: 10 tentativas / 15min em /login
- Bloqueio temporário de IP após N falhas
- CAPTCHA após 3 tentativas consecutivas
- Alerta para tentativas de acesso suspeitas



## Auditoria e Compliance

- Log de toda ação sensível: login, delete, export
- GDPR: consentimento, direito ao esquecimento
- PCI DSS: para armazenar dados de cartão
- SOC 2: para SaaS enterprise — auditoria anual



# Arquitetura de Software

Monolito · Microserviços · Serverless · Event-Driven · Clean Arch · DDD · CQRS · Event Sourcing · Saga

08

## Estilos Arquiteturais — Quando Usar Cada Um

| Estilo           | Como funciona                      | Prós                                | Contras                       | Quando usar                          |
|------------------|------------------------------------|-------------------------------------|-------------------------------|--------------------------------------|
| Monolito         | Uma única aplicação deployada      | Simples, sem latência entre módulos | Escala e deploy acoplados     | MVP, time pequeno, início do produto |
| Modular Monolith | Monolito com módulos bem isolados  | Organização sem complexidade ops    | Deploy ainda é único          | Transição, produto crescendo         |
| Microserviços    | Serviços independentes por domínio | Escala e deploy independentes       | Complexidade operacional alta | Alta escala, times grandes e maduros |
| Serverless       | Funções disparadas por eventos     | Zero infra, pay-per-use, auto-scale | Cold start, vendor lock-in    | ETL, webhooks, tarefas eventuais     |
| Event-Driven     | Comunicação assíncrona via eventos | Desacoplamento, resiliência         | Consistência eventual, debug  | Alta throughput, processamento async |

# Clean Architecture — As 4 Camadas



## Presentation

- Controllers HTTP, resolvers GraphQL, CLI
- Recebe input e retorna response formatada
- Não contém nenhuma lógica de negócio
- Depende apenas da Application Layer



## Application

- Use Cases: orquestram fluxos de negócio
- Chama Domain para regras e Infra para dados
- Um Use Case = uma operação de negócio
- Ex: CreateOrderUseCase, ProcessPaymentUseCase



## Domain

- Regras de negócio puras — ZERO dependências externas
- Entities, Value Objects, Aggregates, Domain Services
- Não conhece frameworks, banco, ou HTTP
- A camada mais valiosa e a mais testável



## Infrastructure

- Implementações concretas: Prisma, Axios, S3, Redis
- Adapters: implementam interfaces definidas no Domain
- Podem ser trocados sem tocar nas outras camadas
- Tests: mockar Infra testa Domain e Application puros

# DDD — Domain-Driven Design



## Entities

- Identidade única (id) — mesmo em estados diferentes
- Ex: User(id=1) é o mesmo mesmo com nome mudado
- Mutáveis — têm ciclo de vida no sistema
- Comparados pela identidade, não pelo valor



## Value Objects

- Sem identidade própria — definidos pelo valor
- Ex: Money(100, BRL) — imutável
- Dois VOs com mesmo valor são iguais
- Ex: Email, Address, DateRange, CPF



## Aggregates

- Cluster de entities com uma Aggregate Root
- Acesso externo apenas pela raiz (Order, não OrderItem)
- Transações dentro de um aggregate
- Consistência garantida dentro do boundary



## Bounded Contexts

- Limite explícito onde o modelo de domínio é válido
- 'User' no Billing ≠ 'User' no Support
- Context Map: relações entre bounded contexts
- Anti-Corruption Layer: isola modelos diferentes



# CQRS e Event Sourcing

## ⚡ CQRS — Command Query Segregation

### Commands alteram estado

CreateOrder, CancelPayment — escrevem no banco principal

### Queries só leem

GetOrderById, ListUserOrders — leem de read models

### Read models otimizados

Desnormalizados para leitura — sem JOINS custosos

### Eventual consistency

Read model atualizado via eventos assíncronos

### Benefício

Escala leitura e escrita de forma completamente independente

## 📖 Event Sourcing

### Estado = sequência de eventos

Nunca UPDATE direto — só APPEND de eventos

### Event Store imutável

Log ordenado e persistente de todos os eventos

### Replay

Reaplica eventos para reconstruir estado em qualquer ponto

### Audit log gratuito

Histórico completo — quem fez o quê e quando

### Usado com CQRS

Eventos disparam atualização dos read models

# Saga, Hexagonal e Circuit Breaker



## Hexagonal (Ports & Adapters)

- Core da aplicação no centro — sem dependências externas
- Ports: interfaces que o core define e expõe
- Adapters: implementações (HTTP adapter, DB adapter, CLI)
- Trocar MySQL por PostgreSQL sem tocar no core



## Saga Pattern

- Transações distribuídas entre microserviços
- Choreography: cada serviço reage a eventos
- Orchestration: Saga Orchestrator comanda os passos
- Compensating transactions para rollback distribuído



## Circuit Breaker

- Evita cascata de falhas em dependências
- States: Closed → Open → Half-Open
- Fallback: dado em cache ou resposta degradada
- Resilience4j (Java), Opossum (Node), Polly (.NET)



## Strangler Fig Pattern

- Migração gradual monolito → microserviços
- Proxy redireciona partes para novos serviços
- Substituição incremental — sem big-bang rewrite
- Reduz risco: volta para o monolito se necessário



# Comunicação Entre Serviços

REST · gRPC · Síncrono · Assíncrono · RabbitMQ · Kafka · SQS · Pub/Sub · Outbox

# Síncrona vs Assíncrona

## ⚡ Síncrona — Request/Response

### REST HTTP interno

JSON sobre HTTP/1.1 — familiar, simples de debugar

### gRPC

Protobuf + HTTP/2 — alta performance, tipo-seguro

### Coupling temporal

Ambos os serviços devem estar UP ao mesmo tempo

### Timeout e retry

Sempre definir timeouts + retry com backoff exponencial

### Ideal para

Queries, validações, operações que precisam de resposta imediata

## ✉ Assíncrona — Mensageria

### Message Queue

Fila de processamento — RabbitMQ, SQS, BullMQ

### Pub/Sub

Publisher → tópico → N subscribers independentes

### Event Streaming

Kafka: log distribuído e durável — replay disponível

### Desacoplamento

Producer e consumer não precisam estar UP juntos

### Ideal para

Background jobs, notificações, pipelines de dados

# Ferramentas de Mensageria



## RabbitMQ

- AMQP — Exchanges, Queues, Bindings
- Routing: direct, topic, fanout, headers
- Dead Letter Queue para mensagens com falha
- Interface de gestão web built-in



## Apache Kafka

- Log distribuído e imutável de eventos
- Throughput altíssimo (milhões de msgs/s)
- Retenção configurável: dias, semanas, forever
- Consumer Groups para escalar consumers



## AWS SQS/SNS

- SQS: managed queue — FIFO e Standard
- SNS: pub/sub → fanout para múltiplos SQS
- EventBridge: event bus serverless + regras
- Serverless-friendly — sem infra para gerenciar



## BullMQ (Node)

- Queue em Redis — priority, delayed, cron
- Retry automático com backoff exponencial
- Dashboard visual: Bull Board
- Ideal para background jobs em apps Node.js

# Padrões de Mensageria



## Pub/Sub

- Publisher emite evento sem conhecer subscribers
- Cada subscriber recebe cópia independente
- OrderPlaced → Email + Inventory + Analytics
- Kafka, SNS/SQS fan-out, Redis Pub/Sub



## Work Queue

- Mensagem consumida por exatamente 1 worker
- Load balancing automático entre consumers
- Processamento garantido — ack após concluir
- Ex: envio de email, resize de imagem, relatório



## Event-Driven Arch

- Serviços emitem eventos ao mudar de estado
- Outros serviços reagem de forma desacoplada
- Event catalog: documentar todos os eventos
- Facilita adicionar novos consumers sem mudar o producer



## Outbox Pattern

- Garante entrega mesmo com falha parcial
- Salvar evento no DB na MESMA transação da mudança
- Worker lê o outbox e publica na fila
- Evita cenário: banco commitou mas fila falhou

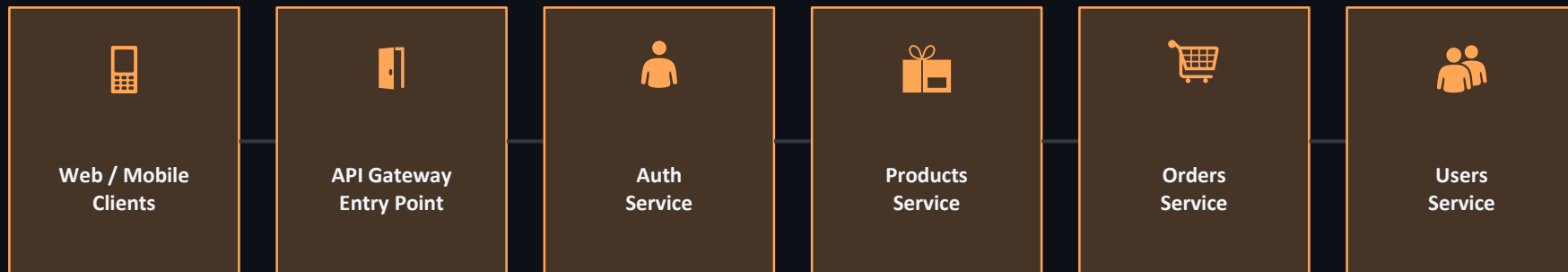


# API Gateway

Roteamento · Auth centralizada · Rate Limiting · Versionamento · Load Balancing ·  
Transformação

10

# API Gateway — Arquitetura Central



- Ponto único de entrada — clientes NUNCA acessam microserviços diretamente
- Autenticação centralizada: JWT validado no gateway — serviços internos confiam no gateway
- Rate limiting centralizado: protege TODOS os serviços de uma vez só
- Load balancing: distribui tráfego entre instâncias do mesmo microserviço



# Funcionalidades do API Gateway



## Roteamento

- `/api/products/*` → Products Service
- `/api/orders/*` → Orders Service
- Path-based, header-based, canary routing
- Rewrite de paths entre cliente e serviço interno



## Auth Centralizada

- Valida JWT antes de encaminhar o request
- Introspection de tokens OAuth 2.0
- API Keys por cliente ou parceiro externo
- Nenhum serviço interno reimplementa validação de auth



## Versionamento

- Roteia `/v1/` → serviço v1 (ainda no ar)
- Roteia `/v2/` → serviço v2 (nova versão)
- Migração gradual sem downtime para clientes
- Deprecation headers para avisar clientes do `/v1/`



## Rate Limiting e Throttling

- Limitar por IP, usuário, API key, plano
- Quotas: free 100/h, pro 10k/h, enterprise  $\infty$
- Spike: burst temporário permitido acima do limite
- Retorna 429 com Retry-After header



## Transformação de Requests

- Modifica headers, body, query params
- SSL termination: HTTPS externo, HTTP interno
- Compressão gzip, caching de respostas
- BFF: agrega chamadas a múltiplos serviços



## Load Balancing

- Round-robin: distribui igualmente
- Least Connections: instância com menor carga
- IP Hash: mesmo cliente → mesma instância
- Health checks: remove instâncias não saudáveis

# Ferramentas de API Gateway

| Ferramenta      | Tipo               | Destaque                                  | Ideal para                          |
|-----------------|--------------------|-------------------------------------------|-------------------------------------|
| Kong            | Open-source / SaaS | Plugins extensíveis, Nginx base, Lua/Go   | Self-hosted, máxima customização    |
| AWS API Gateway | Managed (AWS)      | Integração Lambda/ECS, auth IAM, WAF      | Stack AWS, serverless, pay-per-call |
| Nginx           | Reverse proxy      | Ultra performance, config declarativa     | Simple routing, static files + API  |
| Traefik         | Cloud-native proxy | Auto-discovery Docker/K8s, Let's Encrypt  | Kubernetes, containers, GitOps      |
| Envoy Proxy     | L7 Proxy           | Service mesh ready, xDS, gRPC nativo      | Istio, service mesh, microserviços  |
| Azure API Mgmt  | Managed (Azure)    | Policies XML, developer portal, analytics | Stack Microsoft, enterprise Azure   |



# Infraestructura

Linux · Nginx · Docker · Docker Compose · Kubernetes · Cloud AWS/GCP/Azure · Ambientes

11

# Stack de Infraestrutura Moderna



## Linux

- Ubuntu / Debian — distribuições de servidor
- systemd: gerenciar serviços, logs (journalctl)
- ufw / iptables: firewall — só abrir portas necessárias
- SSH hardening: key-based auth, disable root login



## Nginx

- Reverse proxy de altíssima performance
- SSL termination — HTTPS externo, HTTP interno
- Load balancing upstream entre instâncias
- Serve static files sem passar pelo Node/Python



## Docker

- Container isolado e reproduzível em qualquer host
- Dockerfile: receita de criação da imagem
- Volumes: dados persistentes fora do container
- Networks: comunicação entre containers isolados



## Kubernetes

- Orquestrador de containers em produção
- Pods, Deployments, Services, Ingress, ConfigMaps
- HPA: Horizontal Pod Autoscaler — escala por CPU/RPS
- Helm: package manager para K8s — charts reutilizáveis

# Dockerfile Multi-stage + Docker Compose

```
Dockerfile – multi-stage: imagem final menor e segura
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
RUN addgroup -S app && adduser -S app -G app # non-root
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER app
EXPOSE 3000
HEALTHCHECK --interval=30s CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/main.js"]

docker-compose.yml – ambiente local completo
services:
 api:
 build: .
 ports: ["3000:3000"]
 env_file: .env
 depends_on: { db: { condition: service_healthy } }
 db:
 image: postgres:16-alpine
 healthcheck: { test: ["CMD","pg_isready","-U","postgres"] }
 volumes: [pgdata:/var/lib/postgresql/data]
 redis: { image: redis:7-alpine }
volumes: { pgdata: }
```

## Multi-stage build

Stage builder compila; runner usa só o artefato —  
imagem 3× menor

## Non-root user

Nunca rodar container como root em produção —  
princípio mínimo privilégio

## HEALTHCHECK

Docker e K8s verificam se container está realmente  
saúdável

## depends\_on condition

API aguarda DB estar healthy antes de subir

## Volumes nomeados

Dados do Postgres persistem entre restarts e rebuilds

## env\_file

Secrets fora do código e do Dockerfile — nunca no  
repositório



## Compute e Containers

- EC2 / GCE / Azure VMs — VMs na cloud
- ECS / GKE / AKS — containers managed
- Lambda / Cloud Functions / Azure Fn — serverless
- Auto Scaling Groups — escala automática por demanda



## Banco e Storage Managed

- RDS / Cloud SQL / Azure DB — SQL managed
- DynamoDB / Firestore / CosmosDB — NoSQL
- S3 / GCS / Azure Blob — object storage
- ElastiCache / Memorystore — Redis managed



## IAM e Redes

- IAM: roles e políticas de acesso granulares
- VPC: rede privada isolada na cloud
- Security Groups: firewall por instância
- Private subnets para DBs — nunca expostos à internet



## Ambientes (dev/staging/prod)

- Development: local, hot reload, logs verbosos
- Staging: simula produção, dados sintéticos, testes E2E
- Production: logs estruturados, alertas, otimizações ativas
- Feature flags para promover funcionalidades gradualmente



# DevOps e CI/CD

Cultura · CI/CD · IaC · Terraform · Deploy Strategies · GitOps · IDP · Secrets

12



## Dev + Ops Unidos

- Derrubar silos entre desenvolvimento e operações
- Responsabilidade compartilhada pela produção
- 'You build it, you run it' — Werner Vogels (AWS)
- Feedback rápido: dev vê o impacto do código em prod



## Métricas DORA

- Deployment Frequency: quantas vezes por dia deployam
- Lead Time for Changes: commit → produção (medir em horas)
- MTTR: Mean Time to Restore — quão rápido recuperam
- Change Failure Rate: % de deploys que causam incidente



## Entrega Contínua

- Deploys pequenos e frequentes reduzem risco por deploy
- Feature flags: deploy sem ativar feature para usuários
- Trunk-based development: branch principal sempre verde
- Automação: humano não clica botão de deploy em prod

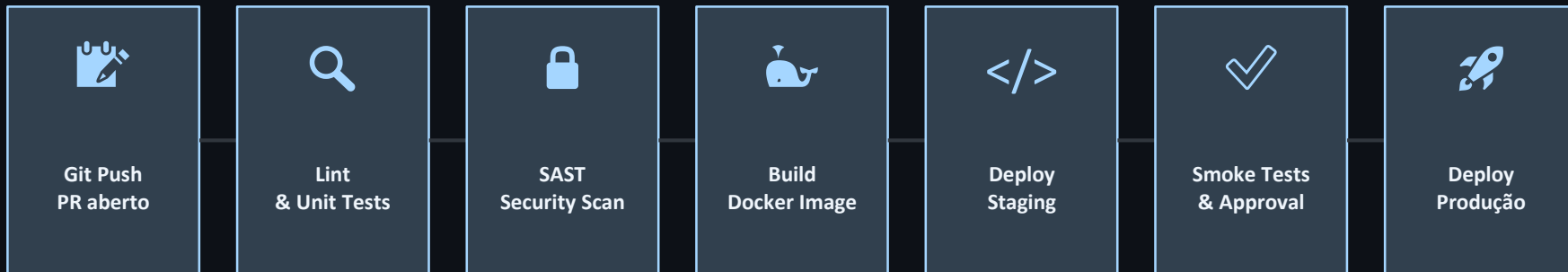


## Feedback Loop

- Alertas automáticos chegam para o time de dev
- Observabilidade em produção — dev vê o que acontece
- Post-mortems blameless — aprender, não punir
- Melhoria contínua do processo via retrospectivas



## Pipeline CI/CD — Do Commit ao Deploy



- Ferramentas CI/CD: GitHub Actions, GitLab CI, CircleCI, Jenkins, Tekton, ArgoCD
- SAST: análise estática de segurança — Snyk, SonarQube, CodeQL detectam vulnerabilidades no código
- Approval gate: aprovação humana obrigatória antes de produção (ou automático com guardrails robustos)
- Rollback automático: smoke tests falharam → pipeline reverte o deploy automaticamente

# IaC — Infrastructure as Code



## Terraform

- HCL declarativo — multi-cloud (AWS, GCP, Azure)
- terraform plan: preview antes de aplicar mudanças
- State file: rastreia recursos criados
- Modules: reutilização de infra como código versionado



## AWS CloudFormation / CDK

- CloudFormation: YAML/JSON nativo AWS
- CDK: IaC em TypeScript, Python, Java sobre CFN
- SAM: serverless framework sobre CloudFormation
- StackSets: deploy em múltiplas contas/regiões



## Azure Bicep

- DSL moderna — evolução do ARM Templates
- Type-safe, modular, compila para ARM JSON
- Azure DevOps + Bicep: CI/CD de infra completo
- Bicep linter: detecta erros antes do deploy



## Ansible

- Configuration management — estado do servidor
- Agentless: só precisa de SSH no target
- Playbooks YAML: instalar software, configurar serviços
- Complementa Terraform: Terraform provisiona, Ansible configura

# Estratégias de Deploy

| Estratégia     | Como funciona                           | Downtime | Rollback                         | Complexidade |
|----------------|-----------------------------------------|----------|----------------------------------|--------------|
| Recreate       | Para tudo e sobe a nova versão          | Sim      | Redeploy da versão anterior      | Baixa        |
| Rolling Update | Substitui instâncias gradualmente       | Não      | Para o rolling, redeploy         | Média        |
| Blue/Green     | 2 ambientes: troca o tráfego de uma vez | Não      | Switch imediato de volta ao Blue | Alta         |
| Canary         | % do tráfego vai para a nova versão     | Não      | Zerar tráfego do canary (0%)     | Alta         |
| Feature Flags  | Toggle de feature em runtime            | Não      | Desabilitar a flag               | Média        |
| Shadow         | Nova versão recebe tráfego espelhado    | Não      | Não precisa — não serve usuários | Alta         |



## GitOps

- Git como ÚNICA fonte de verdade para infra e apps
- ArgoCD / Flux: sincronizam K8s com o repositório
- Pull-based: controller puxa mudanças — não push CI
- Auditoria completa: todo change tem um commit, PR e review



## IDP — Internal Developer Platform

- Portal self-service para desenvolvedores
- Backstage (Spotify): catálogo de serviços e docs
- Provisionar serviço, banco, CI/CD com um click
- Reduz fricção e tempo de onboarding de novos devs



## Secrets Management

- HashiCorp Vault: secrets dinâmicos com rotação automática
- AWS Secrets Manager / GCP Secret Manager
- CI/CD secrets: GitHub Actions, GitLab CI variables
- Nunca em .env commitado — git-secrets previne acidente



## Feature Flags

- Deploy do código sem ativar a feature
- LaunchDarkly, Unleash, Flagsmith, GrowthBook
- Rollout gradual: 1% → 10% → 50% → 100%
- Kill switch: desabilita instantaneamente em produção

# Performance e Escalabilidade

Cache · Redis · CDN · Connection Pool · Load Balancing · Escala Horizontal · Query Optimization



## Cache-Aside

- App verifica cache → miss → DB → popula cache
- Padrão mais usado — controle explícito no app
- Risco: thundering herd em cold start simultâneo
- Solução: cache warming + jitter no TTL



## Write-Through

- Escreve no cache E no DB simultaneamente
- Cache sempre atualizado — sem inconsistência
- Latência de escrita ligeiramente maior
- Ideal: dados lidos frequentemente logo após escrita



## Read-Through

- Cache busca no DB automaticamente em miss
- App só fala com o cache — menos código
- Transparente para a aplicação
- Provider: AWS ElastiCache, Momento, Upstash



## TTL Strategy

- Dados dinâmicos (perfil): 1-5 minutos
- Dados semi-estáticos (produtos): 1 hora
- Dados estáticos (config): 24 horas
- Cache versioning: key inclui version para invalidação



## Cache de Respostas

- Cachear respostas de APIs externas caras
- SETEX key 300 value — expira em 5 minutos
- Reduzir latência e rate limit de parceiros
- JSON.set para objetos complexos (RedisJSON)



## Sessions e Auth

- Session store compartilhado entre instâncias
- Token blacklist para logout de JWT
- Rate limit counters por usuário/IP
- INCR: incremento atômico sem race condition



## Pub/Sub e Filas

- PUBLISH/SUBSCRIBE para notificações simples
- Redis Streams: event streaming leve e durável
- BullMQ: background jobs com retry e prioridade
- Delayed jobs: executar em N segundos/minutos



## Estruturas Avançadas

- ZADD: sorted sets para leaderboards e rankings
- GEO: busca por proximidade geográfica
- Bloom Filter: verificar existência sem ler o DB
- HyperLogLog: contar elementos únicos com ~1% erro



## Vertical vs Horizontal

- Vertical: mais CPU/RAM na mesma máquina — limite físico
- Horizontal: mais instâncias — escala teoricamente infinita
- Horizontal exige: stateless, session store externo, shared DB
- Exemplo Netflix: centenas de instâncias por serviço



## Load Balancing

- Round-Robin: distribui igualmente entre instâncias
- Least Connections: menor carga no momento
- Health checks: remove instâncias doentes
- AWS ELB, GCP Cloud LB, Nginx, HAProxy, Traefik



## CDN

- Edge nodes distribuídos globalmente
- Serve estáticos (JS, CSS, img) do nó mais próximo
- CloudFront (AWS), Cloudflare, Fastly, Akamai
- Reduz latência de 200ms para <20ms para o cliente



## Connection Pooling

- Reutiliza conexões abertas com o banco de dados
- Evita overhead de TCP handshake por query
- PgBouncer: pool externo para PostgreSQL em produção
- HikariCP (Java), asyncpg pool (Python), knex pool (JS)





## Otimização de Queries

- EXPLAIN ANALYZE: plano de execução real da query
- Seq Scan em tabela grande → precisa de índice!
- SELECT só as colunas necessárias — não usar SELECT \*
- Paginação cursor-based vs offset: cursor é muito mais eficiente



## N+1 Problem

- 1 query para listar + N queries para detalhe = N+1
- Solução SQL: JOIN traz tudo em 1 query
- Solução ORM: eager loading (include/joinedload)
- DataLoader: batch e cache de queries em GraphQL



## Stateless Services

- Sem estado local na instância — tudo externalizado
- Sessions em Redis, arquivos em S3, cache em Redis
- Qualquer instância pode servir qualquer request
- Permite escala horizontal sem preocupação com estado



## Database Scaling

- Read Replicas: escala leitura horizontalmente
- Sharding: particiona dados entre instâncias
- CQRS: read model otimizado separado do write
- Antes de escalar o DB: otimizar queries e adicionar índices!

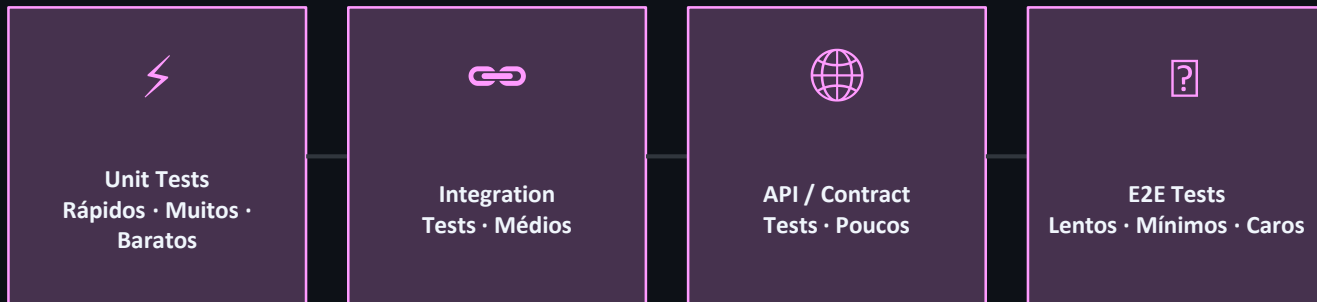


# Testes de Software

Unit · Integration · API · E2E · TDD · Mocks · Stubs · Coverage · Pipeline Automation

# 14

## Pirâmide de Testes — Da Base ao Topo



- Unit: testam função/classe isolada com mocks — rodam em milissegundos — base da pirâmide
- Integration: testam módulos integrados — banco real ou in-memory (SQLite, TestContainers)
- API/Contract: validam endpoints HTTP — Supertest, Postman/Newman, Pact (consumer-driven)
- E2E: fluxo completo no browser — Cypress, Playwright — lentos e frágeis, manter mínimos

# Testes na Prática — Jest + Supertest

```
// Unit Test – isola lógica de negócio
describe('OrderService.calculateTotal', () => {
 let svc: OrderService;
 let mockDiscount: jest.Mocked<DiscountService>;

 beforeEach(() => {
 mockDiscount = { apply: jest.fn() } as any;
 svc = new OrderService(mockDiscount);
 });

 it('aplica 10% para pedidos acima de R$500', async () => {
 mockDiscount.apply.mockResolvedValue(0.1);
 const items = [{ price: 600, qty: 1 }];

 const total = await svc.calculateTotal(items, 'user-1');

 expect(total).toBe(540); // 600 - 10%
 expect(mockDiscount.apply).toHaveBeenCalledWith('user-1', 600);
 });
});

// API / Integration Test – Supertest
describe('POST /api/orders', () => {
 it('cria pedido e retorna 201', async () => {
 const res = await request(app)
 .post('/api/orders')
 .set('Authorization', `Bearer ${testToken}`)
 .send({ items: [{ productId: 'p-1', qty: 2 }] });

 expect(res.status).toBe(201);
 expect(res.body).toMatchObject({
 id: expect.any(String), status: 'pending'
 });
 });
});
```

## beforeEach

Reinicializa estado entre testes — testes sempre independentes

## Mock vs Stub

Mock: verifica se foi chamado. Stub: retorna valor fixo.

## AAA Pattern

Arrange (preparar) → Act (executar) → Assert (verificar)

## TestContainers

Sobe Docker containers reais (PostgreSQL, Redis) em testes

## Test Coverage

Istanbul/nyc: % de linhas cobertas — mínimo 80% em código crítico

## CI Pipeline

Rodar todos os testes em cada PR — bloquear merge em falha



## TDD — Red/Green/Refactor

- Red: escrever teste que falha antes do código
- Green: implementar mínimo para o teste passar
- Refactor: melhorar sem quebrar os testes
- Design emergente — código nasce testável



## Mocks, Stubs e Spies

- Mock: verifica que método foi chamado (comportamento)
- Stub: retorna valor predefinido (controle de input)
- Spy: observa chamadas sem alterar comportamento
- Fake: implementação simplificada (in-memory repository)



## Test Coverage

- Line, Branch, Function e Statement coverage
- 80% mínimo em código de negócio crítico
- Mutation testing: garante que testes realmente verificam
- Não perseguir 100% — cobrir comportamentos, não linhas



## Automação em Pipeline

- Lint + unit tests: em todo commit e PR
- Integration tests: antes de merge na branch principal
- E2E tests: antes de deploy em staging
- Performance tests: antes de releases importantes



# Logs e Observabilidade

Logs · Métricas · Tracing · Prometheus · Grafana · ELK · OpenTelemetry · SLA/SLO/SLI

15

# Os Três Pilares da Observabilidade



## Logs

- Registro de eventos discretos no tempo
- Estruturado (JSON) > texto livre para busca
- Níveis: ERROR > WARN > INFO > DEBUG > TRACE
- Correlation ID: rastrear fluxo entre serviços



## Métricas

- Dados numéricos agregados ao longo do tempo
- RED: Rate, Errors, Duration (para serviços)
- USE: Utilization, Saturation, Errors (para infra)
- Prometheus coleta e armazena time-series



## Distributed Tracing

- Rastreia requests entre múltiplos microserviços
- Trace = sequência de spans com timing de cada hop
- Trace ID propagado via headers por todos os serviços
- Jaeger, Zipkin, AWS X-Ray, Datadog APM



## Prometheus + Grafana

- Prometheus: scraping de métricas via /metrics endpoint
- Exporters: Node Exporter, Redis Exporter, JVM...
- Grafana: dashboards, alertas, correlação de dados
- PrometheusRule: alertas como código (GitOps de alertas)



## ELK Stack

- Elasticsearch: índice e busca full-text de logs
- Logstash / Fluentd / Filebeat: coleta e transformação
- Kibana: dashboards, busca, anomaly detection
- OpenSearch: alternativa open-source ao ELK



## OpenTelemetry

- Padrão aberto vendor-neutral (CNCF)
- SDKs para Node.js, Python, Java, Go, .NET
- Auto-instrumentation: zero código para frameworks populares
- Exporta para Jaeger, Datadog, Grafana Tempo, etc



## Health Checks

- GET /health → 200 OK (processo está rodando)
- GET /health/ready → DB, Redis, deps conectados
- GET /health/live → container pode receber tráfego
- K8s livenessProbe e readinessProbe usam esses endpoints



# Logs Estruturados com Correlation ID

```
import pino from 'pino';
const logger = pino({ level: 'info' });

// Middleware: injeta traceId em todos os logs do request
app.use((req, res, next) => {
 const traceId = req.headers['x-trace-id'] as string || randomUUID();
 req.log = logger.child({
 traceId,
 method: req.method,
 path: req.path,
 userId: req.user?.id,
 });
 res.setHeader('x-trace-id', traceId);
 next();
});

// Uso no service – contexto propagado automaticamente
async function createOrder(req, data) {
 req.log.info({ data }, 'Creating order');
 try {
 const order = await orderRepo.create(data);
 req.log.info({ orderId: order.id, total: order.total }, 'Order created');
 return order;
 } catch (err) {
 req.log.error({ err, data }, 'Order creation failed');
 throw err;
 }
}

// Output: {"level":"info","traceId":"abc-123","userId":"u-1","msg":"Order created"}
```

## JSON estruturado

Campos padronizados: timestamp, level, traceId, service, msg

## Child logger

Herda campos fixos (traceId) — sem repetir em todo log

## Correlation ID

Mesmo traceId propagado por TODOS os serviços — rastrear end-to-end

## Log levels

ERROR: bugs críticos · WARN: degradação · INFO: eventos · DEBUG: dev

## Nunca logar secrets

Mascarar senhas, tokens, PII (CPF, email) nos logs

## Alertas

PagerDuty / OpsGenie acionados por regras  
Prometheus ou ELK



## SLI — Indicator

- Métrica mensurável de performance do serviço
- Ex: % de requests com latência < 200ms
- Ex: % de requests que retornam 2xx
- Ex: disponibilidade medida em janela de 30 dias



## SLO — Objective

- Meta interna de confiabilidade definida pelo time
- Ex: 99.5% dos requests em < 200ms no mês
- Error Budget: 0.5% de 30 dias = 3.6 horas de downtime
- Zerar o budget → congelar features, focar em reliability



## SLA — Agreement

- Contrato EXTERNO com o cliente — penalidade por violação
- Ex: AWS garante 99.99% de uptime no EC2
- SLA deve ser mais conservador que o SLO interno
- Violação → créditos ou reembolso para o cliente



## Alertas e On-call

- Alert fatigue: alertar APENAS para coisas acionáveis
- Runbooks: documentar o que fazer quando alerta dispara
- PagerDuty / OpsGenie: rotação de on-call automatizada
- Post-mortem blameless após qualquer incidente P1/P2

# Integrações Externas

APIs Externas · Webhooks · Pagamentos · E-mail/SMS · Identidade · Sistemas Legados

16

# Tipos de Integração



## Gateways de Pagamento

- Stripe, PagSeguro, Mercado Pago, Adyen
- Webhooks assíncronos para confirmação de pagamento
- Idempotência: nunca cobrar duas vezes o mesmo pedido
- PCI DSS: NUNCA armazenar dados de cartão raw



## E-mail e Notificações

- SendGrid, AWS SES, Mailgun, Resend
- Templates transacionais: boas-vindas, reset de senha
- SMS: Twilio, AWS SNS — verificação por código
- Push notifications: Firebase FCM, Apple APNs



## Sistemas de Identidade

- Auth0, Keycloak, AWS Cognito, Okta
- SSO / SAML: login único corporativo
- Social login: Google, GitHub, Microsoft, Apple
- SCIM: provisionamento automático de usuários



## Sistemas Legados

- SOAP/WSDL: serviços XML antigos (ainda comuns em banco/gov)
- EDI: troca de documentos B2B (850, 856, 810)
- Anti-Corruption Layer: isola o legado do código moderno
- Adapter Pattern: traduz interface antiga para a nova

# Webhook — Implementação Segura e Idempotente

```
// Receber e verificar webhook do Stripe
app.post('/webhooks/stripe',
 express.raw({ type: 'application/json' }), // RAW body obrigatório!
 async (req, res) => {
 const sig = req.headers['stripe-signature'];

 let event;
 try {
 // Verificação de assinatura HMAC — autentica o webhook
 event = stripe.webhooks.constructEvent(
 req.body, sig, process.env.STRIPE_WEBHOOK_SECRET
);
 } catch (err) {
 logger.warn({ err }, 'Invalid webhook signature — possível ataque');
 return res.status(400).send('Invalid signature');
 }

 // Idempotência: mesmo evento pode chegar 2x (Stripe retenta em falha)
 const already = await webhookLog.findOne({ eventId: event.id });
 if (already) return res.json({ received: true, duplicate: true });

 await webhookLog.create({ eventId: event.id });

 // Processar em background — responder 200 RÁPIDO!
 processEvent(event).catch(err =>
 logger.error({ err, eventId: event.id }, 'Webhook processing failed')
);

 res.json({ received: true });
 }
);
```

## Raw body

`express.raw()` necessário — assinatura valida o body cru, não parsed

## HMAC signature

Garante que o webhook veio DO Stripe — não de um atacante

## Idempotência

Webhooks são retentados: processar o mesmo evento 2x não causa duplicata

## Responder 200 rápido

Processamento pesado em background — webhook retenta se demorar >30s

## Dead Letter Queue

Eventos com falha repetida vão para DLQ para análise manual

## Event log

Guardar todos os webhooks recebidos para auditoria e replay



## Retry com Backoff Exponencial

- Retentar em erros transientes: 429, 503, 504
- Aguardar: 1s → 2s → 4s → 8s (+ jitter aleatório)
- Não retentar: 400, 401, 403 — são erros permanentes
- Max retries: 3-5 tentativas com timeout total



## Circuit Breaker

- Detecta falha sistemática na API externa
- Para de chamar temporariamente (open circuit)
- Half-open: testa se o serviço voltou
- Fallback: dado em cache ou resposta degradada



## Cache de Respostas Externas

- Redis cache para responses de APIs caras/lentas
- TTL baseado em frequência de atualização dos dados
- Evita atingir rate limits de parceiros
- Stale-while-revalidate: retorna cache e atualiza async



## SDKs e Contratos

- Usar SDK oficial quando disponível
- OpenAPI → gerar client TypeScript automaticamente
- Monitorar changelog da API externa (breaking changes!)
- Testes de contrato: Pact para consumer-driven contracts



# Boas Práticas Profissionais

Clean Code · SOLID · Code Review · Commits · Semver · Documentação · Manutenibilidade

17

## S

### Single Responsibility

- Cada classe/módulo tem UM motivo para mudar
- UserService só gerencia usuários — não envia email
- Se precisa de 'e' para descrever, são 2 responsabilidades
- Facilita leitura, teste e modificação independente

## O

### Open / Closed

- Aberto para extensão, fechado para modificação
- Adicionar novo pagamento sem alterar OrderService
- Usar interfaces e polimorfismo — não if/switch gigante
- Strategy Pattern é o OCP em prática

## L

### Liskov Substitution

- Subclasse substitui a base sem quebrar o contrato
- Se necessita verificar o tipo concreto — violação de LSP
- Pré-condições não podem ser fortalecidas na subclasse
- Pós-condições não podem ser enfraquecidas

## I

### Interface Segregation

- Interfaces específicas > uma interface gorda
- Readable, Writable, Closable — não um StreamInterface com tudo
- Clientes não devem depender do que não usam
- Muitos métodos na interface → candidato para dividir



## D

### Dependency Inversion

- Dependa de abstrações, não de implementações
- UserService depende de IUserRepository (interface)
- Implementação injetada pelo container (DI)
- Facilita trocar PostgreSQL por MongoDB sem mudar Service

## { }

### Clean Code

- Nomes descritivos: `getUserById()` ✓ vs `getU()` ✗
- Funções curtas: fazem UMA coisa — 20 linhas max
- Números mágicos → constantes nomeadas: `MAX_RETRIES = 3`
- Comentários explicam POR QUÊ, não O QUÊ



### Refatoração Contínua

- Regra do escoteiro: deixar melhor que encontrou
- Dívida técnica: identificar, quantificar, planejar pagamento
- Cobrir com testes ANTES de refatorar — segurança net
- Não refatorar e adicionar feature ao mesmo tempo



### Gestão de Erros

- Erros específicos: `UserNotFoundError` > `Error` genérico
- Nunca `catch {}` vazio — `swallow exception` é bug oculto
- Log com contexto suficiente para debugar em produção
- Nunca expor stack traces ao cliente em produção

# Commits, Semver e Code Review



## Conventional Commits

- feat: nova funcionalidade
- fix: correção de bug
- refactor: sem mudança de comportamento
- docs: / test: / chore: / perf: / ci:
- feat!: ou BREAKING CHANGE no footer



## Semantic Versioning

- MAJOR.MINOR.PATCH → ex: 2.1.3
- MAJOR: breaking changes incompatíveis com versão anterior
- MINOR: novas features compatíveis com versão anterior
- PATCH: correções de bugs compatíveis



## Code Review

- Verificar: lógica, segurança, performance, testes
- PRs pequenos: < 400 linhas são mais efetivos
- Comentários construtivos — revisar o código, não a pessoa
- PR templates: checklist de review automatizado



## Documentação Técnica

- README: setup local, arquitetura, variáveis de ambiente
- ADR: Architecture Decision Records — documentar o porquê
- Runbooks: como operar e remediar incidentes
- OpenAPI: contrato da API sempre sincronizado com o código

# Tendências e Futuro do Backend

AI-First · LLMs · MLOps · AIOps · Edge · Serverless · Zero Trust · GitOps · eBPF · WASM

# 18

## API

- LLMs integrados como componentes do backend
- RAG: Retrieval-Augmented Generation — contexto do seu DB
- Streaming de tokens via SSE para o front-end
- Guardrails: rate limit, content filter, custo por usuário

## MLOps

- Deploy e operação de modelos de ML em produção
- MLflow: tracking de experimentos, versionamento de modelos
- Kubeflow / SageMaker: treino e serving de modelos
- Model drift monitoring: detectar degradação de acurácia

## AIOps

- IA aplicada às operações de TI
- Anomaly detection automática em métricas e logs
- Auto-remediation: corrigir incidentes sem humano
- Root cause analysis automática com LLMs

## AI Agents (MCP)

- LLMs com acesso a ferramentas e APIs reais
- MCP (Model Context Protocol): padrão Anthropic para tools
- Orquestração: LangChain, CrewAI, AutoGen
- Backend como 'sistema nervoso' dos agentes de IA

# Backend com LLM — RAG + Streaming

```
import Anthropic from '@anthropic-ai/sdk';
import { embed, search } from './vectordb'; // pgvector ou Pinecone

const client = new Anthropic();

app.post('/ai/ask', async (req, res) => {
 const { question, userId } = req.body;

 // 1. Gerar embedding da pergunta (vetor numérico)
 const questionEmbedding = await embed(question);

 // 2. Busca semântica no vector DB — contexto relevante
 const context = await search(questionEmbedding, { topK: 5 });

 // 3. Streaming da resposta com contexto via SSE
 res.setHeader('Content-Type', 'text/event-stream');
 res.setHeader('Cache-Control', 'no-cache');

 const stream = client.messages.stream({
 model: 'claude-opus-4-5',
 max_tokens: 1024,
 system: 'Responda baseado APENAS no contexto fornecido.',
 messages: [{
 role: 'user',
 content: `Contexto:\n${context}\n\nPergunta: ${question}`
 }]
 });
});
```

## RAG Pattern

Retrieval-Augmented Generation: contexto do seu próprio banco de dados

## Vector DB

pgvector (PostgreSQL), Pinecone, Qdrant, Weaviate, Chroma, Milvus

## Embeddings

Representação vetorial de texto — busca por SIGNIFICADO, não por palavras

## Streaming SSE

Server-Sent Events: tokens chegam em tempo real no front-end

## MCP Protocol

Model Context Protocol: padrão para tools, recursos e prompts de LLMs

## Cost Control

Rate limit por usuário, max\_tokens, cache de respostas idênticas



## Experimentação

- MLflow: logging de métricas, parâmetros e artefatos
- DVC: versionamento de datasets e modelos (Git para ML)
- Jupyter + Git: notebooks versionados como código
- A/B testing: comparar modelos em produção gradualmente



## Model Serving

- TorchServe, TensorFlow Serving, BentoML
- FastAPI com ONNX Runtime — inference leve e rápido
- Triton Inference Server — GPU serving de alta performance
- Seldon Core / KServe — model serving no Kubernetes



## Monitoramento de Modelos

- Data drift: distribuição dos dados mudou?
- Concept drift: relação input→output degradou?
- Feature importance: quais features mais impactam?
- Evidently AI, Whylogs — monitoring de ML em prod



## CI/CD para ML

- Treinar modelo → validar métricas → aprovar → deploy
- Testes de modelo: acurácia, latência, fairness
- Rollback de modelo se performance cair em prod
- GitHub Actions + DVC + MLflow — pipeline completo



## Anomaly Detection

- IA detecta padrões anômalos em métricas e logs
- Sem threshold manual — aprende o baseline automaticamente
- Datadog Watchdog, Elastic Observability, Dynatrace
- Reduz alert fatigue: foca em anomalias reais



## Auto-Remediation

- Sistema toma ação automática em resposta a alertas
- Ex: pod crashando → restart + notifica dev
- Ex: disco cheio → archiva logs automaticamente
- PagerDuty AIOps, Opsgenie + webhooks, Argo Workflows



## Root Cause Analysis

- Correlaciona eventos de múltiplos sistemas automaticamente
- LLMs analisam logs e sugerem causa raiz
- Reduz MTTR de horas para minutos
- Incident.io, PagerDuty AI, AWS DevOps Guru



## Predictive Scaling

- Prevê picos de tráfego antes de acontecerem
- Pré-aquece infraestrutura com antecedência
- AWS Predictive Scaling, KEDA + custom metrics
- Evita cold start em eventos planejados (Black Friday)

# Edge, Serverless Avançado e Zero Trust



## Edge Computing

- Código executa no PoP mais próximo do usuário
- Cloudflare Workers, Vercel Edge, Deno Deploy
- Latência < 10ms globalmente — impossível com cloud central
- Limitações: sem estado, execução curta, sandbox restrito



## Serverless Avançado

- Durable Functions (Azure): workflows stateful serverless
- AWS Step Functions: orquestração de Lambdas complexa
- Lambda SnapStart: elimina cold start em Java
- Funcionalidades compostas para fluxos multi-step



## Zero Trust Security

- Never trust, always verify — mesmo dentro da rede interna
- mTLS entre todos os micros serviços — identidade mútua
- Service Mesh: Istio, Linkerd gerenciam mTLS automaticamente
- Micro-segmentação: cada serviço tem identidade e políticas



## Data Streaming em Escala

- Kafka como backbone de eventos em tempo real
- Apache Flink: processamento stateful em streaming
- Materialized views em tempo real — dados sempre frescos
- Event-Driven em larga escala: Uber, LinkedIn, Netflix



# eBPF, WebAssembly e GitOps



## eBPF

- Programas no kernel Linux sem módulos customizados
- Observabilidade sem overhead de agents (Cilium, Falco)
- Segurança: network policy, syscall filtering
- Performance profiling em produção sem impacto no app



## WebAssembly (WASM)

- Execução de código compilado no servidor (e no browser)
- Isolamento seguro — mais leve que containers
- Fermion Spin, Wasmtime para WASM servers
- Polyglot sem overhead: Rust, C++, Go → WASM



## GitOps Maduro

- ArgoCD + Flux: sincronização automática K8s↔Git
- Infraestrutura, configs e segredos todos no Git
- Pull Request como gate de deploy para produção
- Auditoria completa: toda mudança tem autor e contexto



## IDP em Evolução

- Backstage: catálogo de serviços + templates + plugins
- Provisionar tudo self-service: app, DB, CI/CD, monit
- Score cards: visibilidade de qualidade por serviço
- Platform Engineering como produto — NPS de devs internos

**Obrigado!**